

Codex Planning Handoff — Free/Open-Source Offline Survival App

0. Purpose of This Handoff

This document captures the implementation-relevant context from the current planning conversation about building a **free/open-source offline survival app**.

The reader should assume they have **not seen the prior chat**. This handoff is intended to be uploaded into Codex alongside two related branch conversations so Codex can continue the project without re-discovering old decisions, making incorrect assumptions, or trying to build the entire product at once.

The goal is not to build another generic survival guide. The goal is to build the best possible **free/open-source survival platform** for outdoors people, preppers, hikers, overlanders, SAR volunteers, CERT groups, emergency responders, and preparedness communities.

1. Project Goal

Core Goal

Build a **free and open-source offline-first survival app** that people would actually install, trust, use in the field, and share.

The product should combine the best parts of:

- Offline survival manuals
- Field tools
- Offline maps
- Checklists
- Preparedness planning
- Content packs
- Citations/provenance
- Optional local AI / RAG later
- Peer-to-peer sharing later
- Community-generated knowledge packs later

The app should be useful **without internet**, **without a cloud account**, and **without AI** in the MVP.

The long-term goal is to become the leading FOSS survival platform, not simply another static manual app.

2. Current State

Product State

No production app has been created in this chat yet.

The work so far is:

- Market research
- Competitor/gap analysis
- Feature prioritization
- Roadmap planning
- Initial architecture decisions
- Codex build strategy
- MVP definition

The project is at the **planning-to-prototype transition point**.

Intended First Platform

The current working assumption is:

- **Android-first**
- Open-source
- F-Droid/GitHub release path
- Mobile-first UX
- Offline-first architecture
- iOS later, not first

Android-first was chosen because:

- Better fit for FOSS distribution
- Easier F-Droid path
- Easier local storage / file import/export
- Better alignment with survival/prepper/outdoor communities
- More flexible for offline maps, sensors, and device variation
- More practical for eventual Meshtastic/ATAK/field-device integrations

3. Strategic Market Context

Important Market Finding

The market gap is **not more survival content**.

There are already many apps and projects with manuals, survival PDFs, field guides, first-aid references, prepper checklists, maps, or offline libraries.

The real gap is:

Integration + trustworthy UX + offline-first practical workflows.

Today's strongest apps usually win one layer at a time:

Product / Category	What It Wins At
Trail Sense	Offline field tools and sensor-based utilities
Organic Maps	Clean offline maps and navigation
OsmAnd	Heavy-duty offline maps, overlays, GPX, routing
Kiwix	Offline libraries and large knowledge archives
Meshtastic	Off-grid radio/mesh communications
Briar	Phone-to-phone encrypted/offline messaging
SurvivalRAG	Cited retrieval over local survival documents
Project NOMAD	Local AI/RAG/offline survival computer concept
CyberDeck-style projects	Offline knowledge + LAN sync + field computing
Static survival guide apps	Lightweight manual access

Very few combine these layers in a polished, lightweight, mobile-first app.

Essentially none combine:

- Offline maps
- Field tools
- Survival checklists
- Pack-based knowledge
- Provenance/citations
- Local search
- Optional AI
- Peer sharing
- Clean consumer-grade UX
- Open-source governance

That combination is the opportunity.

4. Competitor / Inspiration Set

Codex should treat the following as reference points, not things to blindly clone.

4.1 Trail Sense

Relevant traits:

- Open-source Android app
- Strong offline field-tool orientation
- Uses phone sensors well
- Practical outdoor UX
- Small and useful without internet
- Strong example of “field instrument” UX

Features to study:

- Compass
- Weather estimation
- Barometer-style tools where hardware supports them
- Flashlight
- Sun/moon tools
- Ruler / measurement utilities
- Altitude / pressure tools
- Offline-first design

Lesson:

The app should feel like a practical tool, not a generic AI wrapper.

4.2 Organic Maps

Relevant traits:

- Clean offline map UX
- Fast map interaction
- Offline search/navigation
- Good consumer feel

Features to study:

- Offline vector maps
- Search
- Saved places
- Routing

- Low-friction UX
- Minimal clutter

Lesson:

If maps are included, they need to feel fast and trustworthy. Do not create a bad custom map experience if integrating an existing map stack is better.

4.3 OsmAnd

Relevant traits:

- Powerful offline maps
- Topographic overlays
- GPX support
- Route planning
- Advanced map features

Features to study:

- GPX import/export
- Waypoints
- Tracks
- Offline contour/topographic support
- Map layers
- Navigation profiles

Lesson:

OsmAnd defines the advanced outdoor mapping baseline, but its complexity is not the UX model for the MVP.

4.4 Kiwix

Relevant traits:

- Offline libraries
- Large downloadable content archives
- Mature offline knowledge distribution

Features to study:

- Downloadable knowledge bundles
- Offline reading
- Library-style pack management

- Large content management

Lesson:

The app should use optional downloadable packs rather than shipping everything in the base app.

4.5 Meshtastic

Relevant traits:

- Off-grid mesh communication
- Strong prepper/outdoor/hobbyist adoption
- Hardware-dependent

Features to study later:

- Optional message bridge
- Location sharing
- Group communication
- Device pairing
- Offline comms workflows

Lesson:

Meshtastic integration should be delayed. It is powerful but adds hardware dependency and complexity.

4.6 Briar

Relevant traits:

- Offline / peer-to-peer communication
- Privacy-first
- Phone-to-phone sync
- Encrypted communication

Features to study later:

- Local-first messaging
- Peer discovery
- Store-and-forward concepts
- Encrypted local data

Lesson:

P2P features need strong privacy and security design. Do not bolt them on carelessly.

4.7 SurvivalRAG

Relevant traits:

- Local retrieval over survival/medical knowledge
- Cited answers
- More explicit about provenance than most consumer apps

Features to study:

- Source-grounded Q&A
- Chunk-level attribution
- Local document retrieval
- RAG as an optional layer

Lesson:

Provenance-first answers are a major differentiator.

4.8 Project NOMAD

Relevant traits:

- Offline survival computer concept
- Local AI / RAG
- Uses local/server-style components
- Larger-footprint architecture

Features to study:

- Local AI host pattern
- Basecamp/laptop tier
- RAG pipeline structure
- Offline survival workstation concept

Lesson:

Good long-term inspiration, but too heavy for a phone-first MVP.

4.9 Rushlight / CyberDeck-style Projects

Relevant traits:

- Offline knowledge
- Local notes
- Signed or structured field data
- LAN syncing concepts
- Survival computer / field computing mindset

Features to study later:

- Signed field notes
- License sidecars
- Local content sync
- Tamper-evident notes
- Offline LAN sharing

Lesson:

Field intelligence must be provenance-aware, shareable, and tamper-evident if used in serious contexts.

5. Final Product Concept

The strongest product concept is:

A modular offline survival operating layer.

This means the app is not just a guide, not just maps, not just AI, and not just tools.

It is a lightweight mobile field platform with:

- Maps
- Tools
- Guides
- Packs
- Checklists
- Notes
- Waypoints
- Citations
- Optional local retrieval
- Optional local AI
- Optional peer-to-peer sharing
- Optional mesh/LAN integrations later

6. Final Decisions Made

6.1 Start Non-AI

The MVP should be deliberately **non-AI**.

AI is delayed until the app has a useful offline foundation.

Reason:

- Users need immediate utility.
- App size stays smaller.
- Avoids hallucination risk.
- Faster path to public testing.
- Easier F-Droid/GitHub adoption.
- Makes the project useful even on weak phones.
- Avoids the mistake of becoming “just another chatbot.”

MVP should not require:

- Local LLM
- Cloud LLM
- RAG
- Embeddings
- Ollama
- Model downloads
- Account signup

6.2 Android-First

The first build should be Android-first.

Recommended stack:

- Kotlin
- Android native app
- SQLite local storage
- Offline-first data model
- F-Droid-friendly project structure

iOS can come later.

6.3 Four-Tab UX

The app should open into four core tabs:

1. **Map**
2. **Tools**
3. **Guide**
4. **Packs**

Avoid dashboard overload.

The UX principle:

In the field, users need “Where am I?”, “What should I do now?”, and “How do I save/share this?” — not a complex module grid.

6.4 Content Packs Are Central

The core product moat should be the **pack system**.

Packs should be downloadable, importable, versioned, cited, and licensed.

Initial pack format can be simple:

- JSON
- Markdown
- Manifest file
- Local assets if needed

Each content pack should include metadata:

- title
- version
- license
- source_name
- source_url
- last_reviewed
- category
- files
- author
- review_status
- pack_id

This pack model should exist from the MVP, even before AI/RAG.

6.5 Provenance UI Is a Core Differentiator

Every guide card, checklist, first-aid card, or generated answer later should clearly show:

- Source
- License
- Pack version
- Reviewed status
- Last reviewed date
- Whether content is official, community-sourced, imported, or AI-synthesized

The app should distinguish between:

Content Type	UI Treatment
Curated source	High trust, show citation
Community note	Show author/source/community label
Imported user content	Mark as personal/imported
AI answer	Clearly label as synthesis
Model guess	Avoid where possible; never hide uncertainty
Conflicting guidance	Show warning and sources

Key principle:

In an emergency, users need to know whether advice came from a manual, a first-aid protocol, a community note, or a model guess.

6.6 Offline Maps Are Important but Should Be Scoped Carefully

Offline maps are strategically important, but they are also a major implementation burden.

For MVP, do not attempt to build a full Organic Maps/OsmAnd clone.

Start with:

- Current GPS coordinate display
- Saved waypoints
- Waypoint list
- Basic location notes
- Basic map placeholder or minimal offline map integration
- GPX import/export later

The eventual baseline should include:

- Offline vector maps
- Offline search
- Topographic/contour support
- Saved locations
- Breadcrumb/backtrack
- GPX import/export
- On-map notes
- Route/track display

But not all of this belongs in the first sprint.

6.7 Use SQLite-First Local Storage

Storage should be SQLite-first.

Use SQLite for:

- Notes
- Checklists
- Waypoints
- Pack manifests
- Guide cards
- Search index
- User settings
- Installed pack metadata
- Bookmarks
- Local state

Later:

- SQLCipher or equivalent for sensitive data
 - Vector search extension or embedded index
 - FTS5 for keyword search
 - Possible SQLite vector extension depending on platform feasibility
-

6.8 FTS Before Vector Search

For v1 search, use simple local keyword search first.

Recommended progression:

1. Static guide browsing

2. SQLite FTS search
3. Local retrieval with citations
4. Hybrid keyword + semantic retrieval
5. Optional local LLM answer composer

Do not start with vector databases or LLMs in the MVP.

6.9 Local AI Should Be Optional

When added, local AI should be optional and modular.

Operating tiers:

Tier	Target Hardware	AI Role
Phone tier	Normal Android phone	Short answers, summarization, guided Q&A
Tablet/laptop tier	Stronger device	Larger model, richer answers
LAN/basecamp tier	Laptop/local server	Heavy RAG, larger corpora, local model host

Do not hard-wire one model family.

Candidate model classes discussed:

- Phi-3 mini class
- Qwen3-4B class
- Llama 3.1 8B class for laptop/local host
- GGUF models
- Ollama / LM Studio for LAN/basecamp host
- Small local models for phone-tier inference

Important warning:

AI should never be required for core app usefulness.

6.10 RAG Should Be Guided, Not Free-Form Chat First

When RAG is added, start with:

- Guided Q&A
- Source cards
- Retrieved excerpts
- Confidence/provenance UI
- Clear citations

- “Show sources” by default

Avoid launching with a blank free-form chatbot.

Reason:

- Lower hallucination risk
 - Easier trust
 - Easier evaluation
 - Better field usability
 - Easier to constrain responses to curated packs
-

6.11 Community Growth Should Be Pack-Led

The growth loop should not be generic social media.

The growth loop should be:

1. Download app
2. Install a useful pack
3. Use a checklist/guide/map/note in the field
4. Share a pack, checklist, route, or field note
5. Community improves or creates packs
6. More users adopt because the packs match their niche

Target communities:

- Hikers
 - Backpackers
 - Preppers
 - Amateur radio users
 - Overlanders
 - CERT groups
 - Volunteer SAR
 - Homesteaders
 - Disaster-response mutual aid groups
 - Emergency managers
 - Bushcraft communities
 - Off-grid living communities
-

7. Ideas Considered but Rejected or Delayed

7.1 Rejected for MVP: AI-First App

Rejected initial positioning:

“Build an AI survival chatbot first.”

Reason rejected:

- Too easy to become a gimmick
- Trust risk
- Hallucination risk
- Larger app footprint
- More battery drain
- Slower to ship
- Harder FOSS adoption
- Less useful on weak phones

AI is delayed until after the non-AI foundation.

7.2 Rejected for MVP: Free-Form Chatbot

Free-form chatbot UI is delayed.

Instead, later AI should use:

- Guided Q&A
 - Source-grounded answers
 - Confidence indicators
 - Citations
 - Retrieved excerpts
 - Emergency-safe templates
-

7.3 Delayed: Meshtastic Integration

Meshtastic is valuable but delayed.

Reason:

- Hardware dependency
- Complicates onboarding
- Adds Bluetooth/serial/device pairing complexity

- May distract from core survival app usability

Add later as optional bridge.

7.4 Delayed: ATAK Integration

ATAK interoperability is valuable but delayed.

Possible future features:

- Export waypoints
- Import/export routes
- TAK-compatible markers
- Field notes to TAK-style objects
- Plugin or companion app later

Reason delayed:

- Military/government-specific complexity
 - Not required for consumer FOSS MVP
 - Could distort UX too early
-

7.5 Delayed: Full P2P Sync

P2P sync and signed field notes are high-value but delayed.

Future methods:

- QR code
- Bluetooth
- Wi-Fi Direct
- LAN
- Meshtastic bridge
- Export/import files

Reason delayed:

- Conflict resolution complexity
 - Security concerns
 - Tamper-proofing needed
 - UX complexity
-

7.6 Delayed: Large Offline Library Bundles

Do not ship massive content bundles in the base app.

Reason:

- Bloats install size
- Slows onboarding
- Licensing complexity
- Harder updates
- Bad for F-Droid/mobile distribution

Use optional packs instead.

7.7 Delayed: Desktop/Basecamp Version

A desktop/local-host version is useful later, especially for heavy RAG and large corpora.

But the MVP should remain mobile-first.

8. Architecture / Stack Choices

8.1 MVP Architecture

Recommended MVP architecture:

```
Android Kotlin App
├── UI Layer
│   ├── Map tab
│   ├── Tools tab
│   ├── Guide tab
│   └── Packs tab
├── Local Data Layer
│   ├── SQLite database
│   ├── Notes table
│   ├── Checklists table
│   ├── Waypoints table
│   ├── Packs table
│   ├── Guide cards table
│   └── Search index
```

- └─ Pack System
 - └─ Manifest parser
 - └─ JSON/Markdown content parser
 - └─ Local import
 - └─ Pack validation
 - └─ Provenance metadata
- └─ Tools
 - └─ GPS coordinate display
 - └─ Timer
 - └─ Flashlight placeholder/action
 - └─ Compass placeholder/sensor hook
 - └─ Checklist access
- └─ Offline-First Rules
 - └─ No account
 - └─ No cloud dependency
 - └─ No required network
 - └─ Local-first storage

8.2 Suggested MVP Tech Stack

Layer	Recommended Choice
Platform	Android-first
Language	Kotlin
UI	Native Android UI / likely Jetpack Compose if Codex chooses modern Kotlin
Storage	SQLite
Search	SQLite FTS / local keyword search
Content	Markdown + JSON
Pack metadata	Manifest JSON
Maps	Minimal MVP first; offline map library later
AI	None in MVP
Sync	None in MVP
Cloud	None
Distribution	GitHub first, F-Droid path later
License	Apache-2.0 recommended for app core

8.3 Future Retrieval Architecture

Retrieval should evolve in stages:

```
Stage 1: Browse guide cards
Stage 2: SQLite FTS keyword search
Stage 3: Chunked content packs
Stage 4: Local retrieval with citations
Stage 5: Hybrid keyword + semantic retrieval
Stage 6: Optional local answer composer
Stage 7: Optional LAN/basecamp RAG host
```

Potential later tools:

- SQLite FTS5
- SQLite vector extension if practical
- ChromaDB for local desktop/basecamp deployments
- Qdrant for richer local/server footprint
- Local embeddings
- GGUF-compatible models
- Ollama / LM Studio for LAN host mode

8.4 Future AI Architecture

Future AI should be modular:

```
Core App
├─ Works without AI
├─ Local search
├─ Source cards
└─ Optional AI interface

Optional AI Pack
├─ Model manager
├─ Prompt templates
├─ Retrieval context
├─ Answer composer
├─ Citation enforcement
└─ Confidence/provenance UI
```

Do not hard-code one model or runtime.

Supported modes later:

1. No AI
2. On-device small model
3. Tablet/laptop model
4. LAN/basecamp model server
5. External local Ollama/LM Studio endpoint

9. Files, Repo Names, Scripts, Commands, Models, Libraries, APIs, and Tools Mentioned

9.1 Proposed Project / Repo Name

Working title used:

Offline Survival Companion

Alternative positioning:

Offline Survival Companion – Android-first FOSS app with maps, tools, checklists, cited content packs, and optional local AI later.

No final repo name has been locked.

Possible repo names:

offline-survival-companion
open-survival-companion
survival-companion
field-companion
survival-os
offline-field-companion

No repo has been created in this chat.

9.2 Proposed Pack File Concepts

Potential files:

```
pack.json
manifest.json
guide.md
cards.json
checklists.json
sources.json
LICENSE
README.md
```

Suggested pack manifest fields:

```
{
  "pack_id": "wilderness-basics",
  "title": "Wilderness Basics",
  "version": "0.1.0",
  "license": "CC-BY-4.0",
  "source_name": "Example Source",
  "source_url": "https://example.com/source",
  "author": "Pack Maintainer",
  "last_reviewed": "2026-06-30",
  "review_status": "draft",
  "category": "wilderness",
  "files": [
    "guide.md",
    "checklists.json"
  ]
}
```

9.3 Suggested App Repository Files

Codex should create or expect:

```
README.md
LICENSE
CONTRIBUTING.md
CODE_OF_CONDUCT.md
docs/
docs/pack-schema.md
docs/roadmap.md
docs/architecture.md
docs/content-guidelines.md
docs/provenance-model.md
```

```
.github/  
.github/ISSUE_TEMPLATE/  
.github/pull_request_template.md  
app/
```

9.4 Models Mentioned for Later Use

Not for MVP, but relevant later:

```
Phi-3 mini class  
Qwen3-4B class  
Llama 3.1 8B class  
GGUF models
```

Possible host/runtime paths later:

```
Ollama  
LM Studio  
Local LAN model host  
On-device model runtime
```

9.5 Retrieval / Vector Tools Mentioned

Not for MVP, but relevant later:

```
SQLite FTS  
SQLite FTS5  
SQLite vector extension  
ChromaDB  
Qdrant  
Local embeddings  
Hybrid keyword + semantic retrieval
```

9.6 Communication / Sharing Tools Mentioned

Future / delayed:

QR sharing
Bluetooth sharing
Wi-Fi Direct
LAN sync
Meshtastic bridge
ATAK interoperability
GPX import/export

9.7 External Projects Mentioned as Market References

Trail Sense
Organic Maps
OsmAnd
Kiwix
Meshtastic
Briar
SurvivalRAG
Project NOMAD
Rushlight
CyberDeck
LocalForge
The Ark / Survival AI
Prepper AI
ECHO Survival AI
HAVEN
Offline Survival Guide apps
SAS Survival Guide
My Patriot Supply app
Wilderness Survival Guide

Codex should not assume these are dependencies. They are references/inspiration.

10. Constraints, Assumptions, Edge Cases, and Warnings

10.1 Offline-First Is Mandatory

The app should be fully useful without:

- Internet

- Account signup
- Cloud sync
- API key
- Remote model
- Remote database

The MVP should still work in airplane mode.

10.2 AI Is Not the Product

The product is not “a chatbot.”

The product is a field survival platform.

AI may become an optional capability later, but the app must be useful without it.

10.3 Content Trust Is Critical

Survival, first aid, water purification, exposure, shelter, wildfire, disaster response, and emergency content can affect real-world safety.

Therefore:

- Always show source/provenance.
 - Avoid uncited claims.
 - Avoid pretending community content is expert-reviewed.
 - Mark draft/unreviewed content.
 - Mark imported user content.
 - Do not hide uncertainty.
 - Do not generate medical instructions without source grounding.
 - Use quick cards and checklists carefully.
-

10.4 First Aid Content Needs Extra Care

First-aid quick cards are high-value but high-risk.

Codex should not invent medical content.

Use placeholder/example content unless real licensed/cited content is supplied.

Any medical pack should have:

- Source
 - License
 - Review status
 - Last reviewed date
 - Clear disclaimer
 - Emergency services reminder where applicable
 - Local/regional variation awareness
-

10.5 App Size Must Stay Small

Do not bundle massive libraries, models, maps, or PDFs in the base app.

Base app should be lightweight.

Large content should be optional downloads/imports.

10.6 Battery and Thermal Constraints Matter

For field use:

- Avoid constant GPS polling unless user starts tracking.
 - Provide low-battery mode later.
 - Avoid heavy background AI.
 - Avoid constant sensor use.
 - Let user disable features.
 - Cache locally.
 - Minimize wake locks.
-

10.7 Android Device Variance

Android phones vary heavily by:

- RAM
- CPU
- GPU/NPU
- Thermal behavior
- Storage
- Sensor availability
- GPS quality
- Background process limits

- Battery optimization settings

Do not assume every phone has:

- Barometer
 - Compass
 - Reliable GPS indoors
 - Enough RAM for AI
 - Google Play Services
 - Continuous background permissions
-

10.8 Maps Are Expensive to Get Right

Offline maps can become the biggest time sink.

For early MVP:

- Keep map scope minimal.
 - Start with waypoints and coordinate display.
 - Do not block the app on perfect maps.
 - Add full offline vector maps after core pack/guide/checklist UX works.
-

10.9 Licensing Must Be Designed Early

The app core and content packs may need different licenses.

Recommended direction:

Component	Suggested License Strategy
App core	Apache-2.0
Pack schema/spec	Open spec, permissive
Curated content packs	Separate license per pack
Community packs	Must declare license
Imported user content	Local/private unless shared
Maps/data	Respect OSM and provider licenses

Do not assume all survival PDFs/manuals are freely redistributable.

10.10 Provenance Must Be Machine-Readable

Do not make provenance only text in the UI.

Store provenance as structured metadata so it can later support:

- Search filters
 - Trust badges
 - Review status
 - Pack updates
 - Citation cards
 - AI grounding
 - Conflict detection
 - Community moderation
-

10.11 Avoid “Everything App” Failure

The long-term vision includes many features, but the MVP must stay tight.

Do not attempt to build all of these at once:

- Full OsmAnd competitor
- Full Kiwix competitor
- Full Meshtastic client
- Full Briar clone
- Full RAG platform
- Full local AI model manager
- Full ATAK plugin
- Full community registry

The correct move is:

Build a clean offline core, then expand modularly.

11. Exact Plan We Were Converging Toward

Product Direction

Build:

An Android-first, free/open-source, offline survival companion with maps, field tools, checklists, cited content packs, and optional local AI later.

MVP Scope

The MVP should include:

- Four-tab UI
- Local SQLite storage
- Notes
- Checklists
- Waypoints
- Static guide cards
- Content pack installation/import
- Pack manifest metadata
- Provenance UI
- Basic local search
- GPS coordinate display
- Timer
- Flashlight action/placeholder
- Compass placeholder/sensor hook
- No cloud dependency
- No AI dependency
- GitHub-ready repo
- F-Droid-friendly direction

MVP Should Not Include

- Local LLM
- RAG
- Meshtastic
- ATAK
- Full offline vector maps if too slow
- Cloud sync
- User accounts
- Social feed
- Large bundled content
- Paid/subscription logic
- Heavy native model runtime

12. Recommended Roadmap

Phase 0 — Repo + Design Skeleton

Goal:

Create the repository, project skeleton, and documentation.

Deliverables:

- Android Kotlin project
- Four-tab UI shell
- README
- LICENSE
- CONTRIBUTING
- Pack schema documentation
- Roadmap documentation
- Architecture documentation
- Initial issues/tasks

Estimated Codex-assisted time:

1–2 days

Phase 1 — 7-Day Prototype

Goal:

Build a usable non-AI prototype.

Features:

- Four tabs: Map, Tools, Guide, Packs
- Static survival guide cards
- Emergency checklists
- Local notes
- Current GPS coordinate display
- Save/list waypoints
- Manual provenance card UI
- Bundled starter pack
- Local SQLite storage
- Basic importable pack structure

Estimated Codex-assisted time:

2–7 days

Expected quality:

- Demoable
- Not production-ready

- Enough to show direction
 - Enough to start testing with users
-

Phase 2 — 30-Day MVP

Goal:

Build a real open-source MVP people can install and test.

Features:

- Offline content pack system
- SQLite FTS search
- Pack manifest validation
- Guide cards from local packs
- Checklist builder
- Kit checklist templates
- First-aid quick-card structure with citations
- Saved waypoints
- Basic map functionality or map integration
- Export/import packs and checklists
- GitHub release
- F-Droid-ready project path

Estimated Codex-assisted time:

3–6 weeks

Phase 3 — 60–90 Day Public Beta

Goal:

Make it credible as a public FOSS survival app.

Features:

- Better offline maps
- GPX import/export
- Breadcrumb/backtrack
- More polished field UX
- Downloadable/importable packs
- Pack verification status

- Improved provenance UI
- Local retrieval over curated packs
- Source excerpts
- Confidence-aware answers
- No free-form chatbot by default
- Battery mode
- Accessibility/large text mode
- Beta testing with real users

Estimated Codex-assisted time:

8–12 weeks

Phase 4 — RAG / Local AI Version

Goal:

Add grounded local intelligence.

Features:

- Chunked content packs
- Local index
- Keyword + semantic retrieval
- Answer-from-sources mode
- Optional on-device model
- Optional LAN/basecamp model host
- Source-first answer UI
- Model synthesis clearly separated from source excerpts
- Retrieval evaluation tests

Estimated Codex-assisted time:

3–5 months

Phase 5 — Category-Leader Version

Goal:

Become the leading FOSS survival platform.

Features:

- Optional local model tiers
- P2P field-note sharing
- QR sharing
- Bluetooth sharing
- Wi-Fi Direct sharing
- LAN sync
- Signed field notes
- Pack registry
- Community moderation
- Meshtastic bridge
- ATAK import/export
- Multilingual packs
- Accessibility mode
- Formal field testing reports
- Pack verification system
- Contributor ecosystem

Estimated timeline:

6–12 months

13. Codex Build-Time Estimate

Realistic Codex-Assisted Timeline

Goal	Estimated Time
Clickable prototype	2–5 days
Demo you can show people	~1 week
MVP worth open-sourcing	4–6 weeks
Public beta people can test	8–12 weeks
RAG/local AI version	3–5 months
Serious best-in-category contender	6–12 months

Important interpretation:

Codex can generate code quickly, but this project includes:

- UX decisions
- Content licensing
- Offline maps
- Pack design
- Safety review
- Device testing
- Field testing
- Release engineering
- Community building

Those pieces cannot be solved by code generation alone.

14. Immediate Next Steps

Step 1 — Create the Repo

Create the Android project and repo.

Working repo name suggestion:

offline-survival-companion

Use:

Apache-2.0

for the app core unless there is a strong reason to choose GPL.

Step 2 — Build Four-Tab App Skeleton

Tabs:

Map
Tools
Guide
Packs

Each tab should have basic placeholder content and navigation.

Step 3 — Implement SQLite Local Storage

Initial tables:

```
packs
guide_cards
checklists
checklist_items
notes
waypoints
sources
```

Potential later tables:

```
pack_files
bookmarks
search_index
field_notes
sync_events
retrieval_chunks
embeddings
```

Step 4 — Implement Starter Content Pack

Create a bundled starter pack with safe placeholder content.

Do not invent risky medical/survival claims.

Use obvious sample/demo content if real licensed content is not yet available.

Starter pack structure:

```
packs/
  wilderness-basics/
    manifest.json
    guide.md
    checklists.json
    sources.json
```

Step 5 — Build Provenance UI

Every guide card should show:

- Source
- License
- Pack
- Version
- Review status
- Last reviewed date

Even if the first data is placeholder, build the UI pattern now.

Step 6 — Add Local Search

Start with:

SQLite FTS

Search targets:

- Guide card title
 - Body text
 - Category
 - Source name
 - Checklist names
-

Step 7 — Add Basic Field Tools

MVP tools:

- Coordinate display
 - Timer
 - Flashlight
 - Compass placeholder or simple sensor integration
 - Quick access to emergency checklist
-

Step 8 — Add Basic Waypoints

Features:

- Save current coordinate
- Name waypoint
- Add note
- List saved waypoints
- Delete waypoint
- View waypoint details

Map display can be basic at first.

Step 9 — Create Contributor Docs

Docs needed early:

```
README.md
CONTRIBUTING.md
docs/pack-schema.md
docs/content-guidelines.md
docs/provenance-model.md
docs/roadmap.md
```

15. First Codex Prompt to Use

Use this as the initial bounded Codex build prompt:

```
Build an Android-first Kotlin app with four tabs: Map, Tools, Guide, Packs.

Requirements:
- Fully offline-first.
- Use local SQLite storage.
- Store notes, checklists, waypoints, and installed content packs.
- Content packs are local JSON or Markdown bundles with manifest metadata:
  title, version, license, source_url, source_name, last_reviewed, category,
  files.
- Guide tab shows searchable survival cards.
- Each card must show provenance: source, license, reviewed status, and pack
  version.
- Tools tab includes compass placeholder, flashlight action placeholder, timer,
```

coordinate display, and checklist access.

- Map tab initially supports saved waypoints and current GPS coordinate display.
- Packs tab shows installed packs and allows import from local file.
- No cloud dependencies.
- No AI in MVP.
- Prepare repo with README, LICENSE, CONTRIBUTING, pack schema docs, and GitHub issue templates.

16. Suggested Codex Task Breakdown

Task A — Repo Setup

Create Android Kotlin project structure.
Add Apache-2.0 license.
Add README, CONTRIBUTING, roadmap, architecture docs.
Set up basic CI if practical.

Task B — UI Skeleton

Create four-tab navigation:

- Map
- Tools
- Guide
- Packs

Use clean field-readable UI.
Avoid clutter.
Support dark mode if practical.

Task C — Database Layer

Implement local SQLite database.
Create tables for packs, guide cards, checklists, checklist items, notes, waypoints, and sources.
Add repository/data-access layer.
Seed demo data from bundled starter pack.

Task D — Pack Parser

Create local content pack parser.
Support manifest.json.
Support Markdown or JSON guide entries.
Validate required manifest fields.
Store parsed content in SQLite.
Show installed packs in Packs tab.

Task E — Guide Tab

Show guide cards from installed packs.
Add category filters.
Add search.
Each guide card must show provenance metadata.

Task F — Checklist System

Create checklist templates from packs.
Allow users to copy a template into personal checklist.
Allow checking/unchecking items.
Store checklist state locally.

Task G — Notes and Waypoints

Allow user to create local notes.
Allow saving current GPS coordinate as waypoint.
Allow editing waypoint name and note.
Allow deleting waypoint.

Task H — Tools Tab

Implement:
- Timer

- Coordinate display
- Flashlight action if permission/device supports it
- Compass placeholder or real sensor hook
- Quick emergency checklist button

Task I — Search

Add SQLite FTS search over guide cards and checklists.
Keep search fully offline.

Task J — Export/Import

Allow import of local pack file/folder if feasible.
Allow export of user checklists and waypoints as JSON.
Defer P2P sharing.

17. Open Questions / Unresolved Decisions

17.1 Final App Name

Working title:

Offline Survival Companion

Not final.

Need decide:

- App name
- Repo name
- Package name
- Logo/branding direction

17.2 License Choice

Recommended:

Apache-2.0 for app core

But unresolved:

- Whether GPL would better protect openness
 - Whether content packs should use Creative Commons licenses
 - Whether contributor license agreement is needed
 - Whether commercial reuse should be encouraged or restricted
-

17.3 Map Stack

Unresolved.

Options:

- Start with no full map, only coordinates/waypoints
- Use an Android map library
- Integrate OSM-based offline map system
- Support MBTiles
- Use Mapsforge
- Later study Organic Maps/OsmAnd architecture

Codex should not overbuild maps in sprint one.

17.4 Starter Content Sources

Unresolved.

Need decide which survival content is legally usable.

Possible content categories:

- Wilderness basics
- Water
- Fire
- Shelter
- Navigation
- Signaling

- Weather
- Urban emergencies
- Vehicle emergency
- First aid quick cards
- Kit planning

Important:

Do not scrape or bundle copyrighted manuals without permission.

17.5 Medical Content Policy

Unresolved.

Need decide:

- Whether to include first aid in MVP
 - Which licensed sources are safe to use
 - Whether medical packs require review
 - How to label medical content
 - Whether to include emergency disclaimers
-

17.6 AI Runtime

Delayed.

Unresolved:

- Which Android on-device runtime to use
 - Which model family to support first
 - Whether to support GGUF on Android
 - Whether to use external local host first
 - Whether to support Ollama-compatible LAN endpoint
 - How to manage model downloads
 - How to benchmark battery/thermal impact
-

17.7 RAG Index Strategy

Delayed.

Unresolved:

- SQLite vector extension vs custom embedded index
 - Hybrid search implementation
 - Chunk size
 - Embedding model
 - Citation storage
 - Retrieval evaluation harness
 - Confidence scoring
-

17.8 Pack Registry

Delayed.

Unresolved:

- GitHub-based registry
 - Static JSON registry
 - No registry, manual import only
 - Community moderation model
 - Signing/verification model
-

17.9 P2P Sharing

Delayed.

Unresolved:

- QR code first
 - Bluetooth first
 - Wi-Fi Direct first
 - LAN sync first
 - CRDT needed or not
 - Signed field-note format
 - Expiration/TTL for field notes
 - Trust model for received notes
-

17.10 Monetization / Sustainability

The app should be free/open-source.

Unresolved:

- Donations
- GitHub Sponsors
- Paid hosted registry
- Paid commercial support
- Paid verified packs
- Grants
- Consulting
- Dual-license strategy
- Hardware bundles later

Do not add paywalls to the MVP.

18. Features to Include Eventually to Be Better Than the Market

18.1 Core Offline Features

- Fully offline use
 - No account required
 - No cloud required
 - Local-first storage
 - Import/export data
 - Optional pack downloads
 - Airplane-mode functionality
-

18.2 Field Tools

- Compass
 - Flashlight
 - Timer
 - Stopwatch
 - GPS coordinates
 - Altitude where available
 - Barometer/weather estimation where sensors exist
 - Sun/moon data later
 - Signal mirror / whistle patterns as guide cards
 - Battery-aware mode
 - Large text mode
 - Red-light/night mode later
-

18.3 Maps / Navigation

- Saved waypoints
 - Notes tied to waypoints
 - Breadcrumb/backtrack
 - GPX import/export
 - Offline map support
 - Topographic/contour layers
 - Route display
 - Offline search
 - Hazard markers
 - Water/source markers
 - Camp/cache markers
 - Export/share route or waypoint
-

18.4 Guides / Content

- Cited survival cards
 - First-aid quick cards
 - Checklists
 - Kit planning
 - Emergency workflows
 - Wilderness survival pack
 - Urban disaster pack
 - Vehicle emergency pack
 - Homestead/off-grid pack
 - Overlanding pack
 - SAR volunteer pack later
 - Regional packs
 - Multilingual packs
 - Local community packs
-

18.5 Provenance / Trust

- Source card
- License card
- Pack version
- Last reviewed
- Review status
- Community vs official labels
- Imported vs curated labels
- Conflict warnings
- Citation-first UI
- Pack signing later

- Trusted maintainer badges later
-

18.6 Preparedness Planning

- Bug-out bag checklist
 - Vehicle kit checklist
 - Home emergency kit checklist
 - Water storage calculator later
 - Food storage checklist later
 - Family plan templates
 - Rendezvous points
 - Emergency contacts stored locally
 - Local document vault later
-

18.7 Community / Sharing

- Share checklist
 - Share pack
 - Share route
 - Share waypoint
 - Share field note
 - QR export/import
 - Bluetooth/Wi-Fi Direct later
 - LAN sync later
 - Meshtastic bridge later
 - Community pack registry later
-

18.8 AI / RAG Later

- Search local packs
 - Retrieve source excerpts
 - Answer with citations
 - Guided Q&A
 - Confidence indicator
 - "I don't know from installed sources" behavior
 - No hallucinated sources
 - Optional local model
 - Optional LAN model
 - Model tiering
 - Retrieval tests
 - Pack-specific prompts
-

19. Things Codex Must Not Do

Codex should not:

- Build a cloud app.
 - Require login.
 - Start with AI.
 - Require an API key.
 - Bundle a giant model.
 - Bundle huge maps by default.
 - Invent survival/medical content without sources.
 - Hide source/provenance metadata.
 - Build a generic chatbot as the main UX.
 - Attempt Meshtastic/ATAK/P2P in the first version.
 - Overbuild maps before the core app works.
 - Assume all survival content is legally redistributable.
 - Treat community content as verified.
 - Make the app dependent on Google-only services.
 - Break F-Droid compatibility without noting why.
 - Add monetization/paywall logic.
 - Create a bloated dashboard.
 - Assume every Android phone has every sensor.
 - Assume constant GPS polling is acceptable.
 - Assume AI can run well on every phone.
-

20. Definition of Done for the First Prototype

The first prototype is complete when:

- Android app launches.
- Four tabs exist: Map, Tools, Guide, Packs.
- App works without internet.
- SQLite stores local data.
- Bundled starter pack loads.
- Guide cards display from local pack.
- Guide cards show provenance metadata.
- User can create/check checklist items.
- User can create local notes.
- User can view current GPS coordinate if permission granted.
- User can save a waypoint.
- Packs tab shows installed pack metadata.
- README explains the project.
- Pack schema doc exists.
- No AI is included.

- No cloud dependency exists.
-

21. Definition of Done for the MVP

The MVP is complete when:

- Users can install and use the app offline.
 - Users can browse local survival content.
 - Users can search content offline.
 - Users can manage checklists.
 - Users can save notes.
 - Users can save waypoints.
 - Users can import at least one local content pack.
 - Every content item has visible provenance.
 - Project has clear open-source docs.
 - Project has a public roadmap.
 - Project can accept contributor-created packs.
 - App does not require AI, login, cloud, or API keys.
 - App is ready for real-world beta feedback.
-

22. Final Strategic Takeaway

The winning product is not:

Another survival manual app

and not:

An AI survival chatbot

The winning product is:

A free/open-source, offline-first survival companion that combines practical field tools, maps, checklists, cited content packs, and optional local intelligence into one clean mobile experience.

The app should earn trust first.

Then add retrieval.

Then add AI.

Then add sharing.

Then add mesh/community integrations.

The first version should be small, useful, offline, cited, and easy to contribute to.

That is the path to pulling the market in a meaningful way.